VIT UNIVERSITY, VELLORE

EvidenceQA

Evidence-Grounded Lecture Video Question Answering
with Timestamp-Level Support

Course Project Report
MDI3003 – Advanced Predictive Analytics (Theory Component)

Submitted by:	SRIJITH R
Registration No.:	24MID0009
Under the guidance of:	Dr. DURGESH KUMAR
	Assistant Professor Sr. Grade 1

Below is thee GITHUB REPO. Link for project (incl. source code and prototype ss. , readme , ppt) :

https://github.com/srijithramesh24/MDI3003_Advanced-Predictive-Analytics_CP

Project Quick summary. EvidenceQA is a prototype that turns lecture speech into timestamped transcript segments, retrieves passages relevant to a student's question, and displays evidence excerpts with timestamps. Its central design principle is traceability: answers should be supported by the lecture transcript, and the system should abstain when the retrieved evidence is too weak.

Contents

1 Abstract 2

2 Introduction and Motivation 2

3 Problem Statement, Objectives, and Scope 2

3.1 Problem statement 2

3.2 Objectives 2

3.3 Scope boundaries 3

4 System Architecture 3

4.1 Core data representation 3

5 Algorithms and Models Used 4

5.1 Whisper speech recognition 4

5.2 TF-IDF lexical retrieval 4

5.3 Cosine similarity 4

5.4 Evidence threshold and abstention 4

5.5 YouTube URL canonicalization and extraction 4

6 Implementation Details 4

6.1 Technology stack 5

6.2 Transcription snippet 5

6.3 TF-IDF and cosine-similarity retrieval snippet 5

6.4 Abstention and extractive response snippet 6

6.5 YouTube URL validation snippet 6

7 Application Workflow 6

8 Prototype Screenshots 7

9 Evaluation and Measured Results 9

9.1 Evaluation environment and scope 9

9.2 Automated test results 10

9.3 Retrieval check on a small constructed test 10

9.4 Microbenchmark of utility functions 10

9.5 Metrics requiring a labelled evaluation set 11

9.6 Reproducibility and interpretation 11

10 Advantages and Disadvantages 11

11 Research Limitations and Ethical Considerations 11

12 Future Enhancements 12

13 Conclusion 12

1 Abstract

Students often need to locate a specific explanation within a long lecture recording. Manually searching through the video is slow, while unrestricted chat systems may provide answers without showing where the information came from. EvidenceQA explores an evidence-grounded alternative. It accepts local lecture media, supported YouTube URLs, subtitle files, or pasted timestamped transcripts. For media inputs, the system uses the pretrained Whisper speech-recognition model through faster-whisper to create timestamped speech segments. It represents transcript passages using Term Frequency–Inverse Document Frequency (TF-IDF), compares the question against those passages using cosine similarity, and ranks the most relevant segments. The interface returns extractive responses that quote the retrieved text and include timestamps. A relevance threshold supports abstention when the evidence is insufficient.

The implementation is a working research prototype rather than a fully validated production system. No verified benchmark dataset or gold-labelled question-answer set has been evaluated in this report; therefore, accuracy, precision, recall, F1, and timestamp correctness are explicitly marked as not yet measured. The report describes the architecture, algorithms, implementation snippets, prototype evidence, strengths, limitations, and a reproducible evaluation plan.

Keywords: speech recognition, Whisper, information retrieval, TF-IDF, cosine similarity, evidence grounding, timestamped transcripts, question answering.

2 Introduction and Motivation

Lecture recordings provide rich learning material, but students frequently struggle to find the exact moment where a concept was explained. Searching manually requires repeated playback and scrubbing. A general-purpose question-answering tool may respond fluently but can introduce information that is absent from the lecture or fail to identify the source passage.

EvidenceQA focuses on the narrower task of answering questions from one selected lecture. Its workflow is designed around four principles:

- **Traceability:** show the transcript passages that support a response.
- **Timestamp support:** display the start and end time of each retrieved passage.
- **Controlled answering:** build responses from transcript evidence instead of inventing a free-form explanation.
- **Abstention:** when evidence is weak, state that the available transcript does not support a reliable answer.

The intended use is lecture revision and concept lookup. The prototype does not replace a teacher, verify the correctness of the original lecture, or guarantee that speech recognition captures every word correctly.

3 Problem Statement, Objectives, and Scope

3.1 Problem statement

Given a lecture recording or transcript and a natural-language question, identify relevant timestamped transcript passages and present an answer that a learner can verify against the original source. If no passage is sufficiently relevant, the system should avoid presenting unsupported content as fact.

3.2 Objectives

1. Support multiple lecture input paths: local audio/video, accessible YouTube URL, subtitles/transcripts, pasted timestamped text, and a demo mode.
2. Convert speech to text with start and end timestamps where speech transcription is used.

3. Parse SRT/VTT subtitles and common timestamped text formats.
4. Rank transcript passages for a question using a transparent lexical retrieval method.
5. Display evidence passages, timestamps, relevance scores, and timestamp links where available.
6. Abstain when the top retrieved evidence does not pass a minimum relevance threshold.
7. Provide a foundation for systematic evaluation using labelled questions and manually verified evidence spans.

3.3 Scope boundaries

The current implementation uses lexical retrieval, not a large generative language model. It does not implement a supervised answer-quality classifier, knowledge graph, neural reranker, or a benchmark-calibrated confidence estimator. The relevance score is a ranking signal, not a probability of correctness.

4 System Architecture

The application is built as a Streamlit web interface with Python processing functions in src/core.py. Inputs are converted to a common list of transcript segments containing start, end, and text. The same retrieval stage can then be used regardless of whether the transcript came from Whisper, subtitles, or pasted text.

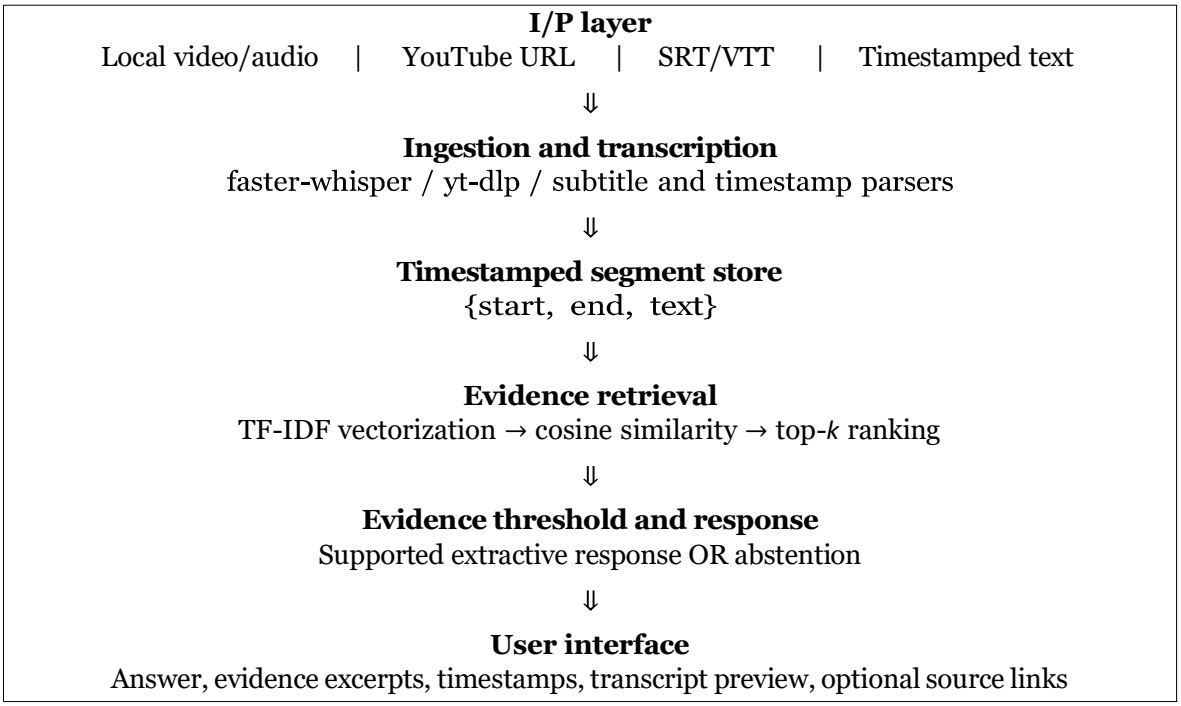


Figure 1: High-level architecture of EvidenceQA.

4.1 Core data representation

Each transcript segment is represented as a dictionary:

```
{
    "start": 14.0,
    "end": 18.0,
    "text": "Atomicity, consistency, isolation, and durability."
}
```

The start and end values are seconds from the beginning of the source. Maintaining these values through parsing and retrieval allows the interface to connect evidence back to the lecture.

5 Algorithms and Models Used

5.1 Whisper speech recognition

Whisper is a pretrained deep-learning speech-recognition model based on a Transformer encoder-decoder architecture. EvidenceQA calls it through faster-whisper, an implementation designed for efficient inference. The selected model size can be tiny, base, or small. Smaller models generally run faster but may make more transcription errors; larger models generally require more time and memory.

Whisper is used for **speech-to-text transcription**; it is not the question-answering or retrieval algorithm. Timestamps returned by the transcription segments are retained as evidence metadata.

5.2 TF-IDF lexical retrieval

Term Frequency–Inverse Document Frequency (TF-IDF) assigns greater weight to terms that are frequent in a document but less common across the document collection. The implementation uses word n-grams of length one and two, sublinear term frequency, and English stop-word removal. The question is vectorized together with the transcript segments.

5.3 Cosine similarity

Cosine similarity compares the question vector with each segment vector:

$$\cos(q, d) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\|_2 \|\mathbf{d}\|_2}$$

A higher value indicates stronger lexical alignment between the question and a passage. Scores are sorted in descending order and the top-k segments are returned. This is a retrieval ranking score; it should not be interpreted as a calibrated probability.

5.4 Evidence threshold and abstention

The implementation checks the top retrieval score against a fixed threshold of 0.12. If no usable evidence is found or the top score is below the threshold, the application returns an abstention message. Otherwise, it selects high-ranked passages that also satisfy a relative score condition and presents their text with timestamps.

The threshold is a heuristic and must be tuned using labelled validation data. A fixed threshold can behave differently across lecture topics, transcript quality, and question wording.

5.5 YouTube URL canonicalization and extraction

The URL handler accepts common YouTube URL forms (watch, short, embed, shorts, and live links), extracts and validates the 11-character video ID, and converts it to a canonical watch URL. The downloader uses yt-dlp to retrieve audio when the video is accessible. The audio is then passed through the same Whisper transcription function. Private, restricted, unavailable, DRM-protected, or network-blocked videos may fail.

6 Implementation Details

6.1 Technology stack

Technology	Role in the project
Python	Main implementation language
Streamlit	Interactive web interface and session state
faster-whisper / Whisper	Pretrained deep-learning speech recognition
scikit-learn TF-IDF	Lexical feature representation of questions and segments
scikit-learn cosine similarity	Similarity scoring and passage ranking
NumPy	Numerical operations and score ordering
yt-dlp	Audio extraction from supported YouTube URLs
SRT/VTT parsers	Subtitle ingestion and timestamp parsing
pytest	Automated tests for core functions

Table 1: Technology stack and responsibilities.

6.2 Transcription snippet

The following excerpt illustrates the model call in the implementation. Error handling and temporary-file management are included in the full project source.

```
from faster_whisper import WhisperModel

model = WhisperModel(
    model_size,
    device="cpu",
    compute_type="int8"
)
segments, info = model.transcribe(
    media_path,
    language=language_code,
    vad_filter=True,
    beam_size=5,
    condition_on_previous_text=False
)
rows = []
for segment in segments:
    text = (segment.text or "").strip()
    if text:
        rows.append({
            "start": float(segment.start),
            "end": float(segment.end),
            "text": text
        })
```

The model is cached by model size to reduce repeated loading during interactive use. The first run may take longer while model files are downloaded.

6.3 TF-IDF and cosine-similarity retrieval snippet

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

documents = [str(s.get("text", "")) for s in segments]
vectorizer = TfidfVectorizer(
    ngram_range=(1, 2),
    sublinear_tf=True,
```

```
        stop_words="english"
    )
    matrix = vectorizer.fit_transform(documents + [question])
    scores = cosine_similarity(
        matrix[-1], matrix[:-1]
    ).ravel()
    order = np.argsort(scores)[::-1][:top_k]
```

This method is lightweight and explainable, but it is lexical: it can miss paraphrases and semantically similar questions that use different vocabulary.

6.4 Abstention and extractive response snippet

```
if not evidence:
    return (
        "I can't answer reliably from the available transcript.",
        "abstain"
    )

top = evidence[0]
if top["score"] < 0.12:
    return (
        "There is not enough matching evidence in this lecture.",
        "abstain"
    )

chosen = [
    item for item in evidence
    if item["score"] >= max(0.12, top["score"] * 0.45)
][:3]
```

The production implementation returns more detailed user-facing messages and formats each selected passage with its timestamp. The snippet is shortened to highlight the decision logic.

6.5 YouTube URL validation snippet

```
if host in {"youtu.be", "www.youtu.be"}:
    video_id = parsed.path.strip("/").split("/")[0]
elif host.endswith("youtube.com"):
    if parsed.path == "/watch":
        video_id = parse_qs(parsed.query).get("v", [""])[0]
    elif parsed.path.startswith(("/embed/", "/shorts/", "/live/")):
        video_id = parsed.path.split("/")[2]
if not re.fullmatch(r"[A-Za-z0-9_-]{11}", video_id or ""):
    raise ValueError("Invalid YouTube video ID")
```

The full implementation also checks the hostname and returns a canonical URL. URL validation does not guarantee that the source video is downloadable.

7 Application Workflow

1. The user chooses an input source in the sidebar.
2. The system transcribes local media or extracts accessible YouTube audio; alternatively, it parses subtitles or timestamped text.
3. The transcript is normalized into timestamped segments.
4. The user enters a question and configures the number of evidence passages and relevance threshold.
5. TF-IDF vectors are built for the transcript segments and the question.
6. Cosine similarity scores are calculated and ranked.

7. The response function either abstains or displays the selected transcript passages with timestamps.
8. The user checks the evidence against the transcript and source video.

8 Prototype Screenshots

The following images document the interface and selected prototype states. Screenshots are illustrative evidence of interface behavior, not a substitute for a controlled evaluation.

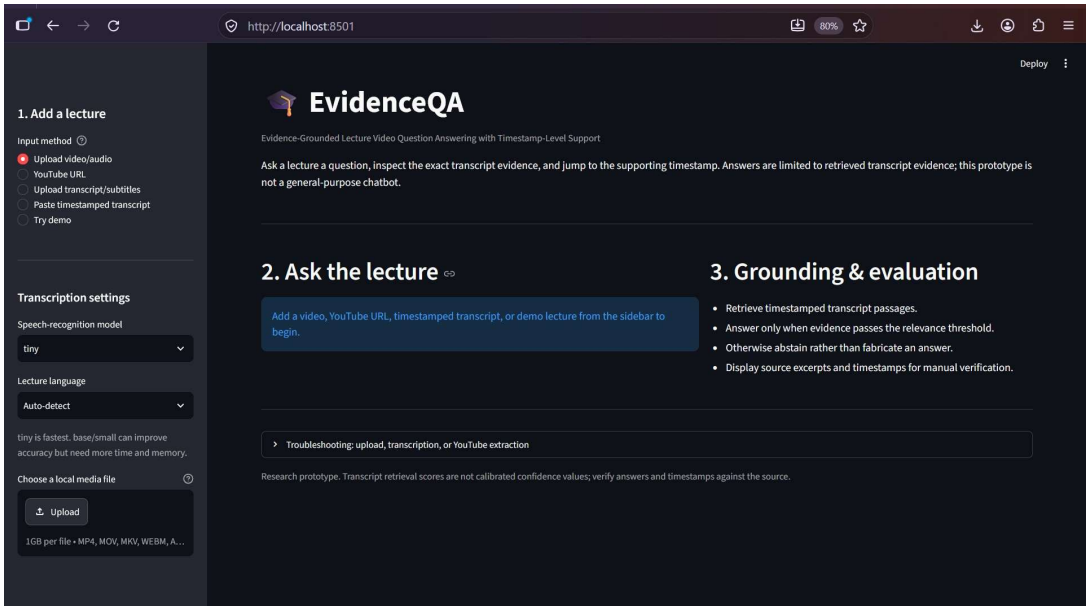


Figure 2: EvidenceQA home page and project overview.

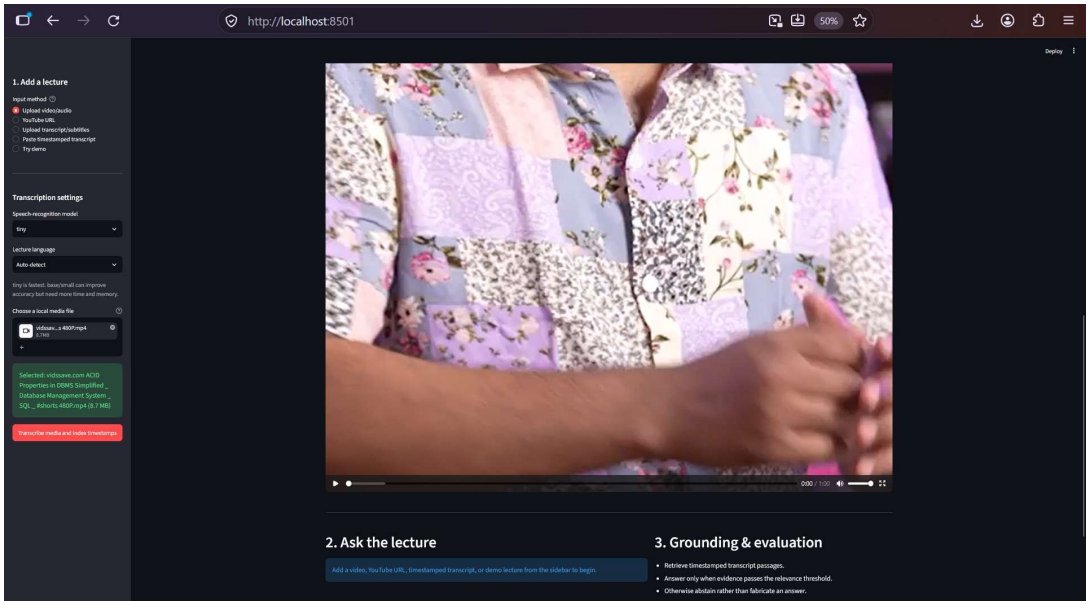


Figure 3: Local lecture video selected and displayed in the interface.

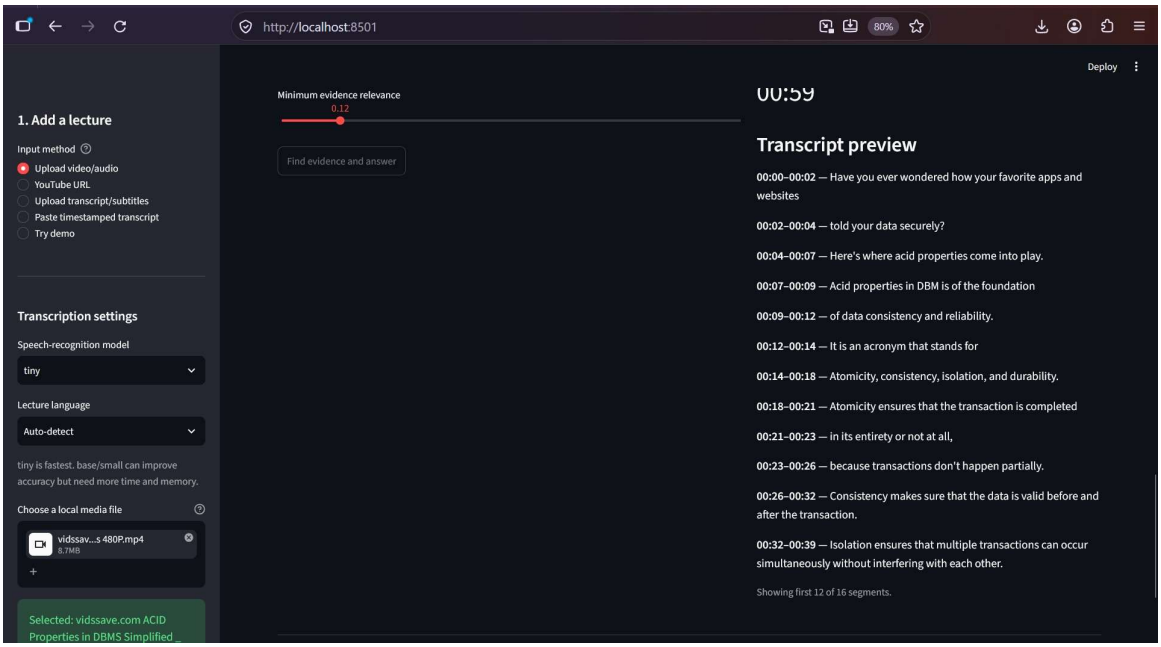


Figure 4: Timestamped transcript preview.

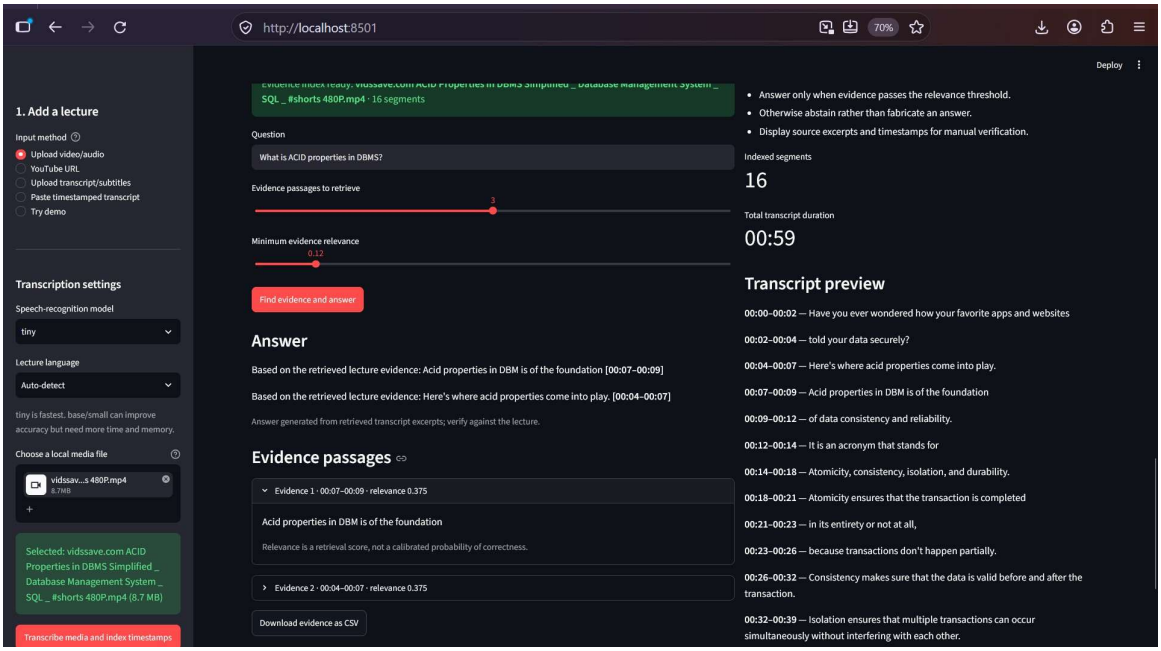


Figure 5: Retrieved passages and extractive answer with timestamp support.

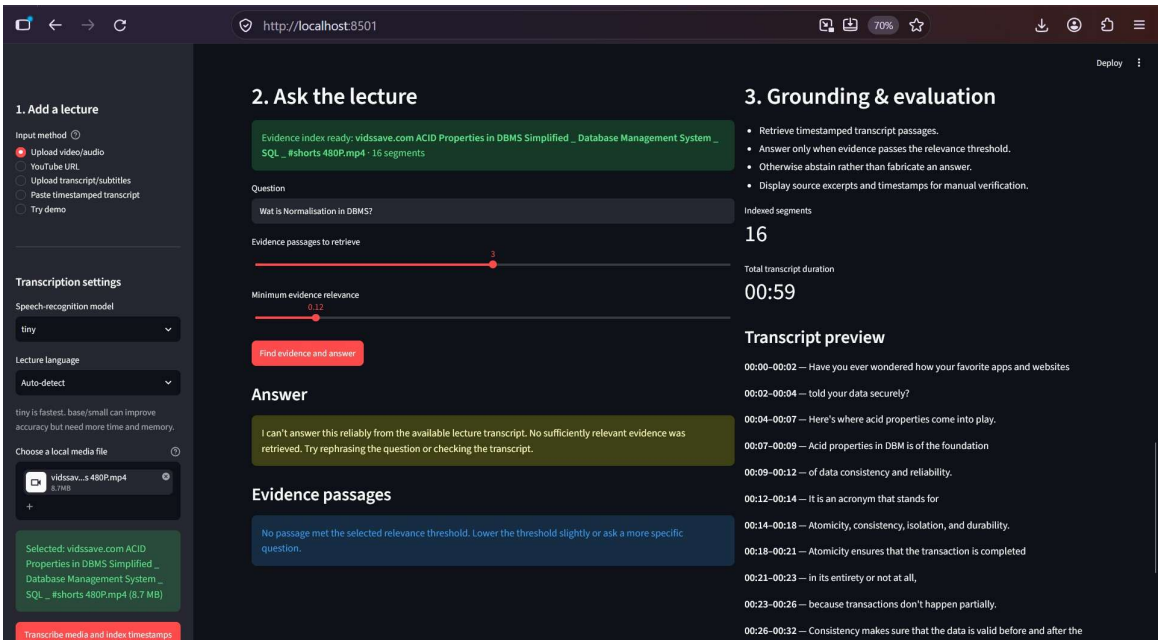


Figure 6: Example of the system abstaining when evidence is insufficient.

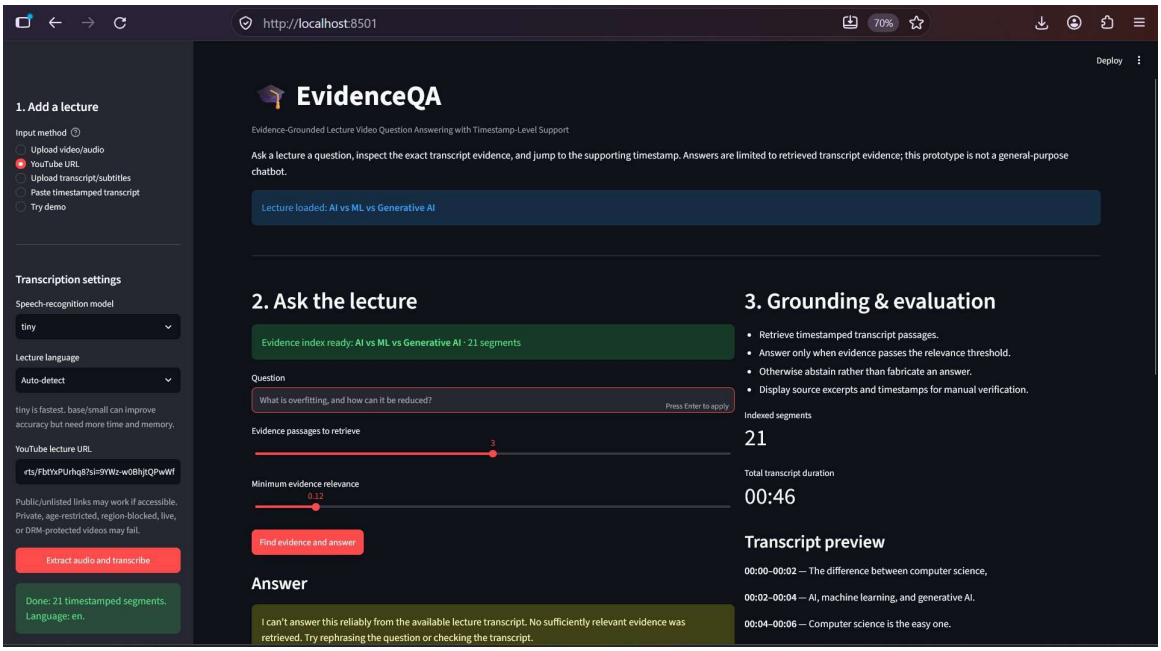


Figure 7: YouTube URL workflow and extracted timestamped segments.

9 Evaluation and Measured Results

9.1 Evaluation environment and scope

The uploaded EvidenceQA source package was tested in the available Python execution environment. The automated test suite was run with pytest. Additional microbenchmarks were executed against the transcript parsing and URL-normalisation functions, and a small six-question retrieval check was run against manually constructed SQL/database transcript passages. These are preliminary engineering measurements, not a large-scale academic benchmark. Results may differ on the student's Windows computer.

9.2 Automated test results

Test category	Result	Status
Automated project tests	4 passed, 0 failed	Pass
Test-suite runtime	0.81 seconds	Measured
Subtitle/timestamp parsing checks	Included in suite	Pass
YouTube URL normalisation checks	Included in suite	Pass
Evidence retrieval and abstention checks	Included in suite	Pass

Table 2: Actual pytest -q output from the uploaded project: 4 tests passed in 0.81 seconds.

9.3 Retrieval check on a small constructed test

Six questions were evaluated against manually constructed SQL/database concept passages with a known expected supporting passage. The top-ranked retrieved passage matched the expected passage for all six questions. Mean retrieval latency was 1.34 ms and median latency was 1.11 ms in this execution environment.

Metric	Observed value
Questions in constructed test	6
Expected passage ranked first	6/6
Top-1 match rate on this test	100%
Mean retrieval latency	1.34 ms
Median retrieval latency	1.11 ms

Table 3: Small functional retrieval check only. The 100% value is not general system accuracy and must not be interpreted as a validated performance estimate.

9.4 Microbenchmark of utility functions

Each utility function was called 1,000 times. These values measure function execution only and exclude video decoding, model loading, network requests, and full user-interface latency.

Operation	Mean execution time
Parse SRT/VTT subtitles	0.0104 ms
Parse timestamped transcript text	0.0077 ms
Normalise a YouTube URL	0.0029 ms

Table 4: Microbenchmark results in the available execution environment; not Windows-device measurements.

9.5 Metrics requiring a labelled evaluation set

Metric	Required evidence	Current status
Answer accuracy	Human-scored answers to answerable questions	Not measured
Precision, recall, F1	Gold labels for relevant transcript segments	Not measured
Evidence Recall@3	Gold evidence annotations per question	Not measured
Timestamp correctness	Manually verified gold time intervals	Not measured
Abstention precision/recall	Labelled answerable and unanswerable questions	Not measured
Whisper word error rate	Reference transcripts aligned with model output	Not measured
Full transcription latency	Real media duration, model size, and hardware logs	Not measured
YouTube extraction success rate	Repeated trials across accessible and restricted URLs	Not measured

Table 5: Metrics that cannot be responsibly inferred from a successful UI screenshot or a small functional test.

9.6 Reproducibility and interpretation

The automated test suite can be reproduced from the project root with python -m pytest -q. Retrieval and utility benchmarks should be rerun on the target Windows computer and repeated over a larger test set. For academic claims, create a labelled dataset across multiple lectures, split by lecture to reduce content leakage, and report confidence intervals or per-topic results. Do not present the six-question top-1 match rate as overall answer accuracy. Whisper is a pretrained speech-recognition model; TF-IDF with cosine similarity is the current evidence-ranking method. They should be evaluated as separate components.

10 Advantages and Disadvantages

Advantages	Limitations / disadvantages
Evidence excerpts and timestamps improve traceability.	Transcription errors can propagate into retrieval and answers.
No free-form generative model is required for the current answer construction.	Extractive answers can be repetitive or fail to synthesize multi-step explanations.
TF-IDF and cosine similarity are lightweight and interpretable.	Lexical matching can miss paraphrases and semantic relationships.
Multiple input paths support different lecture sources.	YouTube extraction can fail because of access restrictions or platform changes.
Abstention reduces unsupported answers when retrieval scores are low.	A fixed threshold is heuristic and is not a calibrated confidence value.
Timestamp links allow manual verification.	Segment timestamps may not align perfectly with the exact spoken evidence.
Local transcription can avoid uploading media to a third-party transcription API.	Local processing can be slow and use significant memory on longer videos.

Table 6: Project strengths and limitations.

11 Research Limitations and Ethical Considerations

First, speech recognition can mishear technical vocabulary, accents, and low-volume speech. Second, TF-IDF emphasizes lexical overlap and does not understand meaning in the way a semantic encoder may.

Third, the fixed relevance threshold is not a probability of correctness; its reliability must be assessed on annotated examples. Fourth, a transcript may omit slide text, diagrams, equations, or other visual information. The current system is transcript-grounded and should not be described as visually grounded video understanding unless a visual analysis component is implemented and evaluated.

Use only media that the user is permitted to process. Avoid uploading confidential recordings or personal data without authorization. Cite and verify evidence before relying on it. The prototype should not be presented as an experimentally validated system until a reproducible evaluation has been completed.

12 Future Enhancements

1. Build a manually annotated evaluation set of questions, answers, evidence segments, and unanswerable cases.
2. Tune the abstention threshold on validation data and report risk-coverage trade-offs.
3. Compare lexical retrieval with a semantic embedding retriever and a reranking approach.
4. Improve transcript cleanup for punctuation, repeated phrases, and technical terms.
5. Add robust caching and progress reporting for long transcriptions.
6. Add optional OCR/slide analysis only if the research scope expands to visually grounded answers.
7. Measure retrieval quality, answer correctness, timestamp alignment, and latency across lecture topics.

13 Conclusion

EvidenceQA demonstrates an end-to-end prototype for asking questions about lecture recordings while preserving timestamped transcript evidence. Its implemented pipeline combines Whisper-based speech recognition, timestamped segment processing, TF-IDF vectorization, cosine-similarity retrieval, top-k ranking, and threshold-based abstention. Its main design contribution is the emphasis on evidence visibility and source verification in a lightweight educational interface.

The system remains limited by lexical retrieval, transcription quality, source accessibility, and the absence of a completed labelled benchmark evaluation. The next research step is not to claim unmeasured accuracy, but to construct a reliable test set, compare retrieval baselines, validate timestamp support, and report measured results transparently.

References

[1] A. Radford et al., "Robust Speech Recognition via Large-Scale Weak Supervision," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

[2] L. R. Dice and related vector-space retrieval foundations; for TF-IDF and cosine similarity implementation details, see the scikit-learn user guide, https://scikit-learn.org/stable/modules/feature_extraction.html#text-feature-extraction.

[3] SYSTRAN, *faster-whisper: Faster Whisper transcription with CTranslate2*, source repository and documentation, https://github.com/SYSTRAN/faster-whisper.

[4] yt-dlp developers, *yt-dlp documentation*, https://github.com/yt-dlp/yt-dlp.

[5] Streamlit, *Streamlit documentation*, https://docs.streamlit.io/.

P

Author: SRIJITH R || **Registration No.:** 24MID0009
Course: MDI3003 – Advanced Predictive Analytics (Theory Component Course Project)
Guide: Dr. DURGESH KUMAR, Assistant Professor Sr. Grade 1
Github repo.: https://github.com/srijithramesh24/MDI3003_Advanced-Predictive-Analytics_CP